Trabajo Practico Final

Integrantes:

- Facundo della vedova
- Eitan Zarate
- Fernando Zarazola

Índice

1. Introducción
2. Métodos implementados
3. Código fuente métodos
4. Muestra de funcionamiento en Swagger
5. Propuestas
6. Conclusión

Introducción

Esta presentación fue realizada con el fin de completar el proceso de desarrollo realizado, en los siguientes puntos se encuentran los métodos desarrollados, las pruebas que verifican el funcionamiento adecuado de los métodos, las propuestas desarrolladas por el grupo para la mejora del proyecto y una conclusión final que recopila lo que ha sido el trabajo realizado.

Métodos implementados

El trabajo realizado consistió en el desarrollo de los siguientes métodos planteados:

- Agregar: Inserta un nuevo producto al catálogo, creando las categorías intermedias que sean necesarias.
- Buscar: Permite realizar una búsqueda de productos en donde se muestran los que cumplan la coincidencia con la cadena introducida
- Todos: se obtienen todos los productos almacenados
- GetUrlsSeo: se obtienen todas las URL amigable de los productos almacenados
- GetUrlSeoPorId: se obtiene la URL amigable del producto que posee el id introducido
- ConsultaNiveles: devuelve los elementos del catálogo organizados por nivel

Código fuente métodos

Método Agregar y su auxiliar EncontrarHijo

```
public void Agregar(ArbolGeneral<ItemCat> arbol, ItemCat dato, string rutaAlPadre)
        {
            List<string> ruta = new List<string>(rutaAlPadre.Split('/'));
            ArbolGeneral<ItemCat> actual= arbol;
            int i=0;
            while (i<ruta.Count())
            {
                ArbolGeneral<ItemCat> siguiente=encontrarHijo(actual,ruta[i]);
                if (siguiente== null)
                {
                    break;
                }
                i++;
                actual=siguiente;
            }

            for (int x = i; x < ruta.Count(); x++)
            {
                ItemCat nodo= new ItemCat(ruta[x],TipoElemento.Categoria);
                ArbolGeneral<ItemCat> nuevo= new ArbolGeneral<ItemCat>(nodo);
                actual.agregarHijo(nuevo);
                actual=nuevo;
            }

            actual.agregarHijo(new ArbolGeneral<ItemCat>(dato));
        }
private ArbolGeneral<ItemCat> encontrarHijo(ArbolGeneral<ItemCat> arbol,string dato)
        {
            foreach (ArbolGeneral<ItemCat> hijo in arbol.getHijos())
            {
                if (hijo.getDatoRaiz().Nombre==dato)
                {
                    return hijo;
                }
            }
            return null;
        }
```

Método Buscar y su auxiliar _Busqueda

```
public List<ItemCat> Buscar(ArbolGeneral<ItemCat> arbol, string elementoABuscar)
        {
            List<ItemCat> resultados=new List<ItemCat>();
            if (arbol.esHoja()) {
                return resultados;
            }else{
                _busqueda(arbol,elementoABuscar.ToLower().Trim(),resultados);
            }

            return resultados;
        }
private void _busqueda(ArbolGeneral<ItemCat> arbol,string elemento,List<ItemCat>
resultados)
        {
            foreach (ArbolGeneral<ItemCat> X in arbol.getHijos()) {
                if (X.getDatoRaiz().Tipo== TipoElemento.Producto) {
                    if ((X.getDatoRaiz() != null) &&
(X.getDatoRaiz().Nombre.ToLower().Trim().Contains(elemento))) {
                        resultados.Add(X.getDatoRaiz());
                    }
                }else{
                    _busqueda(X,elemento,resultados);
                }
            }
        }
```

Método Todos y su auxiliar agruparProd

```
public List<ItemCat> Todos(ArbolGeneral<ItemCat> arbol)
        {
            List<ItemCat> productos=new List<ItemCat>();
            if (arbol.esHoja()) {
                return productos;
            }else{
                agruparProd(arbol,productos);
            }

            return productos;
        }

private void agruparProd(ArbolGeneral<ItemCat> arbol,List<ItemCat> productos)
        {
            foreach (ArbolGeneral<ItemCat> X in arbol.getHijos()) {
                if (X.getDatoRaiz().Tipo== TipoElemento.Producto && X.getDatoRaiz() !=
             null) {
                            productos.Add(X.getDatoRaiz());
                }else{
                    agruparProd(X,productos);
                }
            }
        }
```

Método GetUrlsSeo y su método auxiliar recorrido

```
public List<string> GetURLsSEO(ArbolGeneral<ItemCat> arbol)
        {
            List<string> urls=new List<string>();
            string url=""+arbol.getDatoRaiz().Nombre.Trim();
            if (arbol.esHoja()) {
                return urls;
            }else{
                recorrido(arbol,urls,url);
            }

            return urls;
        }

private static void recorrido(ArbolGeneral<ItemCat> arbol,List<string> urls,string url)
        {
            foreach (var x in arbol.getHijos()) {

                if (x.getDatoRaiz().Tipo==TipoElemento.Producto ) {
                    urls.Add(url+"/"+x.getDatoRaiz().Nombre.Trim());
                }else{
                    recorrido(x,urls,url+"/"+x.getDatoRaiz().Nombre.Trim());
                }
            }
        }
```

Método GetUrlSeoPorId y su auxiliar RecorriidoSeoPorId

```
public string GetUrlSeoPorId(ArbolGeneral<ItemCat> arbol, int id)

        {
            if (arbol.esHoja())
            {
                return arbol.getDatoRaiz().Nombre.Trim();
            }
            else
            {
                string url = arbol.getDatoRaiz().Nombre.Trim();
                return RecorridoSeoPorId(arbol, id, url);
            }
        }

private string RecorridoSeoPorId(ArbolGeneral<ItemCat> arbol, int id, string url)
        {
            foreach (var x in arbol.getHijos())
            {
                string nuevaUrl = url + "/" + x.getDatoRaiz().Nombre.Trim();

                if (x.getDatoRaiz().Tipo == TipoElemento.Producto)
                {
                    if (x.getDatoRaiz().Id == id)
                    {
                        return nuevaUrl;
                    }
                }
                else
                {
                    string resultado = RecorridoSeoPorId(x, id, nuevaUrl);

                    if (resultado != "")
                    {
                        return resultado;
                    }
                }
            }

            return "";
        }
```

Método ConsultaNiveles y su auxiliar recorrido_nivel

```
public List<List<string>> ConsultaNiveles(ArbolGeneral<ItemCat> arbol)
        {
            List<List<string>> niveles=new List<List<string>>();
            if(arbol.esHoja())
            {
                List<string> aux=new List<string>();
                aux.Add(arbol.getDatoRaiz().Nombre);
                niveles.Add(aux);
                return niveles;
            }
            else
            {
                recorrido_nivel(arbol,niveles);
            }
            return niveles;
        }

private void recorrido_nivel(ArbolGeneral<ItemCat> arbol,List<List<string>> niveles)
        {

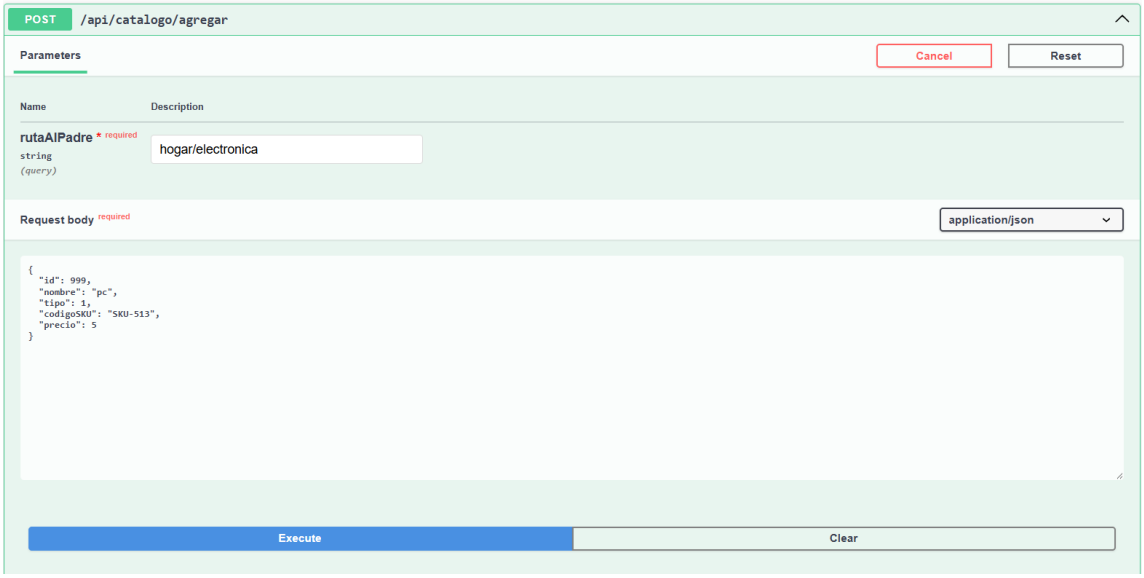
            Cola<ArbolGeneral<ItemCat>> c= new Cola<ArbolGeneral<ItemCat>>();
            ArbolGeneral<ItemCat> aux;

            c.encolar(arbol);
            while (!c.esVacia())
            {
                int cant=c.cantidadElementos();
                List<string> l_aux=new List<string>();

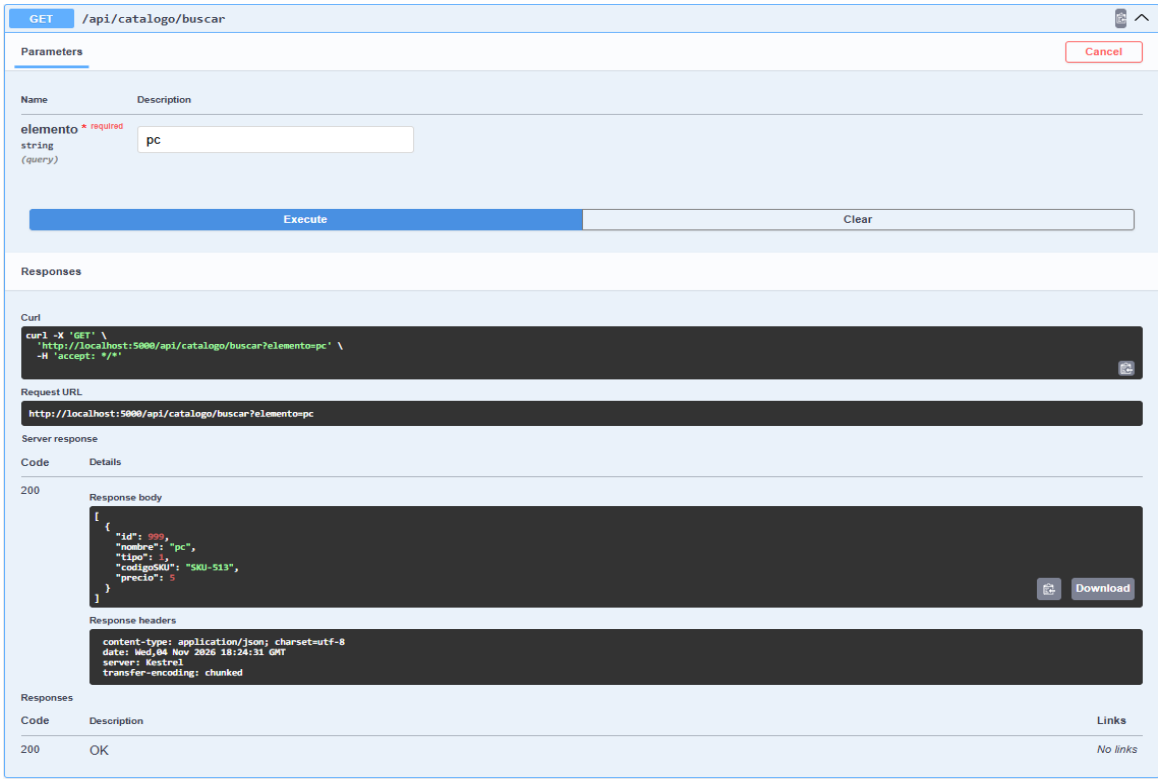
                for (int i = 0; i < cant; i++)
                {
                    aux=c.desencolar();
                    l_aux.Add(aux.getDatoRaiz().Nombre);
                    foreach (var hijo in aux.getHijos())
                    {
                        c.encolar(hijo);
                    }
                }
                niveles.Add(l_aux);
            }
        }
```

Muestra funcionamiento en Swagger

Utilizando el método agregar agregamos un nuevo producto con las siguientes características:

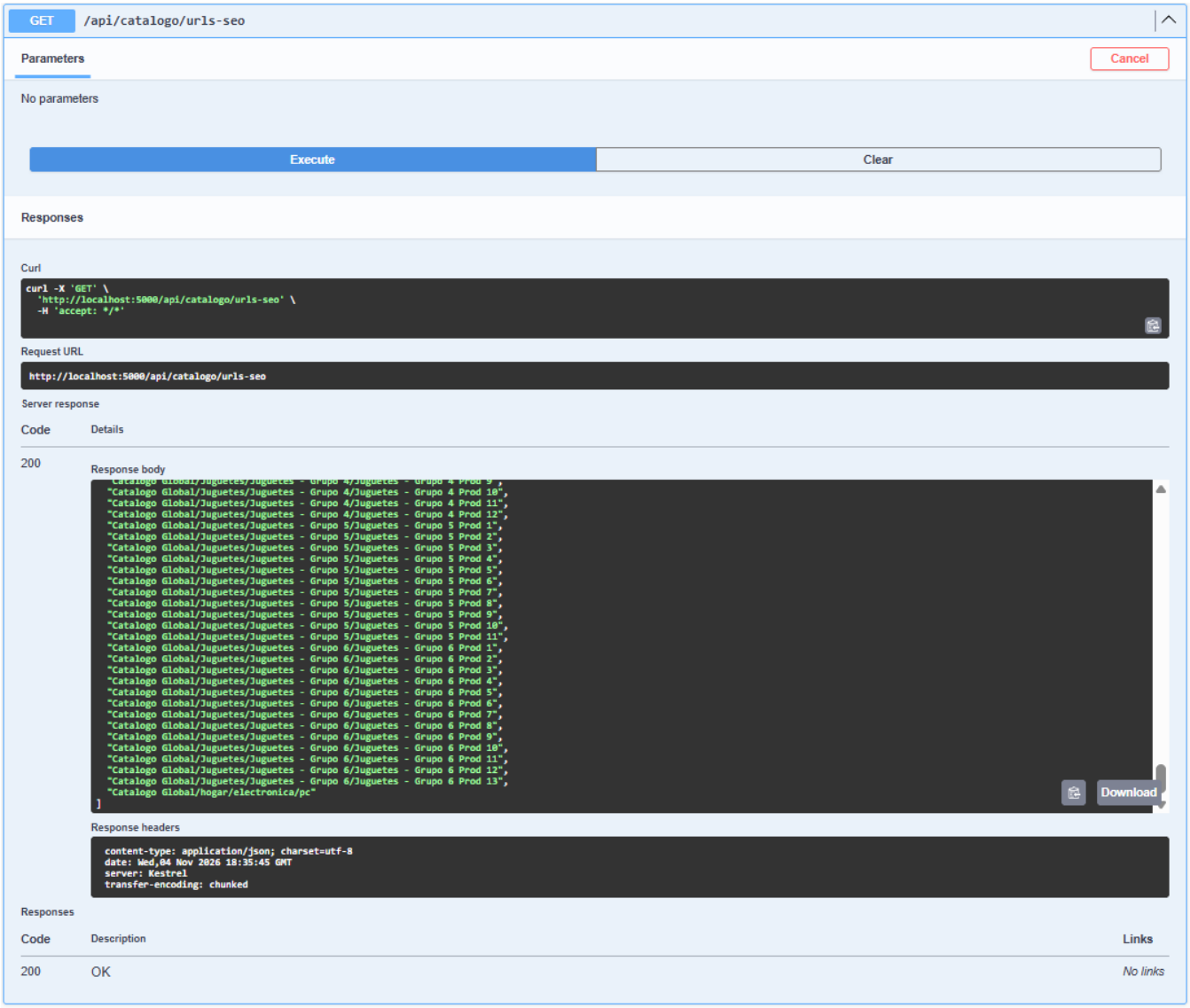


Podemos verificar que se haya creado satisfactoriamente utilizando el método Buscar, al ingresar el nombre del producto que agregamos nos los devuelve

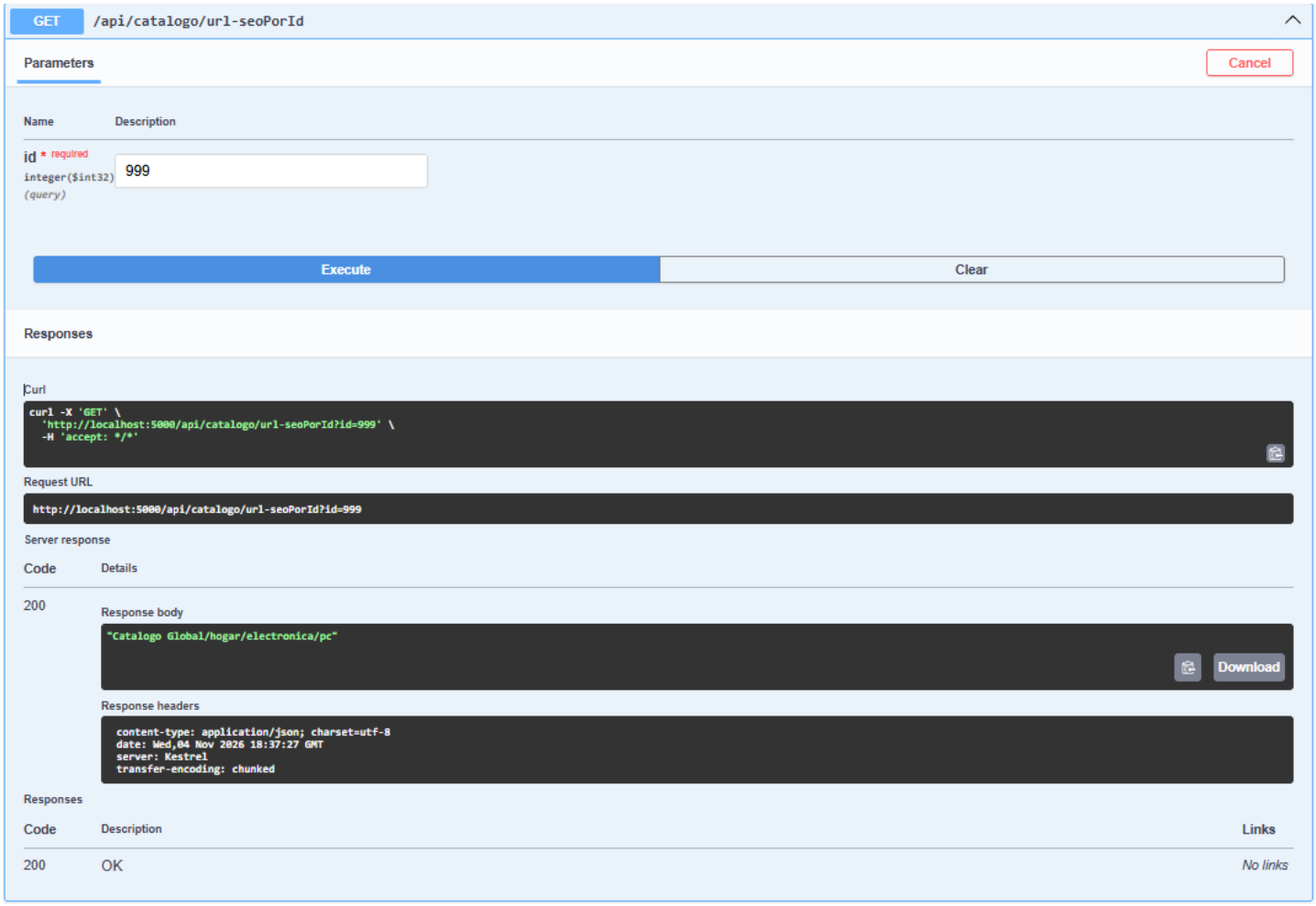


El método buscar devuelve una lista de productos que coincidan parcial o totalmente con la cadena ingresada como parámetro de busqueda, en este caso al solamente existir un elemento que contenga como nombre pc, es el único que obtenemos.

Utilizando el método GetUrlsSeo obtenemos todas las urls de los productos en el catálogo



Utilizando el método GetUrlsSeoPorId, ingresamos el id del producto que queremos obtener su url, en este caso utilizando el mismo producto creado en el primer método obtenemos su UrlSeo



Por último, quedan los métodos Todos y ConsultaNiveles. Todos devuelve una lista con todos los productos almacenados en el catalogo y ConsultaNiveles devuelve una lista que contiene una lista individual para cada nivel que posee el catálogo, los niveles contienen categorías y productos indiscriminadamente.

GET /api/catalogo/todos

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5000/api/catalogo/todos' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5000/api/catalogo/todos
```

Server response

Code Details

200

Response body


```
  },
  {
    "id": 337,
    "nombre": "Juguetes - Grupo 6 Prod 11",
    "tipo": 1,
    "codigoSKU": "SKU-9520",
    "precio": 208.48
  },
  {
    "id": 338,
    "nombre": "Juguetes - Grupo 6 Prod 12",
    "tipo": 1,
    "codigoSKU": "SKU-2605",
    "precio": 72.66
  },
  {
    "id": 339,
    "nombre": "Juguetes - Grupo 6 Prod 13",
    "tipo": 1,
    "codigoSKU": "SKU-4201",
    "precio": 382.08
  },
  {
    "id": 999,
    "nombre": "pc",
    "tipo": 1,
    "codigoSKU": "SKU-513",
    "precio": 5
  }
]
```


Download

GET /api/catalogo/niveles

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5000/api/catalogo/niveles' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5000/api/catalogo/niveles
```

Server response

Code Details

200

Response body


```
[
  [
    "Catalogo Global"
  ],
  [
    "Electrónica",
    "Moda",
    "Hogar",
    "Deportes",
    "Belleza",
    "Juguetes",
    "hogar"
  ],
  [
    "Electrónica - Grupo 1",
    "Electrónica - Grupo 2",
    "Electrónica - Grupo 3",
    "Electrónica - Grupo 4",
    "Electrónica - Grupo 5",
    "Electrónica - Grupo 6",
    "Moda - Grupo 1",
    "Moda - Grupo 2",
    "Moda - Grupo 3",
    "Moda - Grupo 4",
    "Hogar - Grupo 1",
    "Hogar - Grupo 2",
    "Hogar - Grupo 3",
    "Deportes - Grupo 1",
    "Deportes - Grupo 2",
    "Deportes - Grupo 3"
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,04 Nov 2026 18:45:36 GMT
server: Kestrel
transfer-encoding: chunked
```

Propuestas

Como posibles propuestas presentamos:

- Optimización de URLs SEO: se propone modificar el método GetURLsSEO para que, en lugar de devolver una lista de cadenas (List<string>), devuelva un Dictionary<ItemCat, string>, donde cada producto (ItemCat) sea la clave y tenga asociada una cadena que represente el url.
  La implementación continuaría realizando un recorrido en preorden del árbol, pero, en lugar de agregar las URLs a una lista, las almacenaría en el diccionario, asociándolas al producto correspondiente.
  De esta manera, el método GetUrlSeoPorId podría localizar el producto que posea el identificador solicitado y obtener directamente sus URLs mediante el acceso al diccionario, evitando realizar un segundo recorrido en preorden para recuperar las URLs correspondientes.
  Esta modificación permitiría optimizar la obtención de URLs y aprovechar las ventajas de la estructura de datos Dictionary.

- Incorporación de búsquedas por criterios específicos: utilizando el método Todos, se podrían implementar métodos auxiliares que permitan realizar búsquedas según diferentes condiciones, como obtener productos cuyo precio sea mayor a $100.
  Para ello, se podría aplicar un algoritmo de ordenamiento sobre la lista de productos, organizándolos según su precio. Posteriormente, mediante una búsqueda binaria, se podría encontrar el primer producto cuyo precio sea igual o superior a $100 y, a partir de esa posición, recorrer los elementos restantes para obtener aquellos que cumplan con la condición.
  Esto permitiría ampliar las funcionalidades del catálogo y aprovechar los algoritmos de ordenamiento y búsqueda estudiados durante la materia.

Conclusión

Con este ultimo punto finaliza el recorrido por el desarrollo realizado en este trabajo, durante este tiempo hemos aprendido nuevas habilidades y herramientas, tanto para poder afrontar el desarrollo del trabajo como para poder comunicarnos entre los integrantes de este grupo.

En un principio, el proyecto parecía abrumador debido a su complejidad. Sin embargo, realizando un trabajo en conjunto, solucionando dudas, realizando pruebas sobre lo aprendido durante la cursada e implementado, consideramos que hemos llegado a un punto en donde hemos podido cumplir en gran medida las expectativas que teníamos cuando iniciamos este trabajo.

El trabajo se desarrolló tomando las opiniones de cada integrante, viendo diferentes acercamientos a las implementaciones que cada uno diseño, probando cada método para tratar de hallar el mas completo o que mejor se adaptase a los requerimientos solicitados. Creemos que este trabajo nos ha beneficiado al poder generar un ambiente realista en el cual hemos tenido que dejar de lado nuestros intereses personales y priorizar los objetivos grupales.